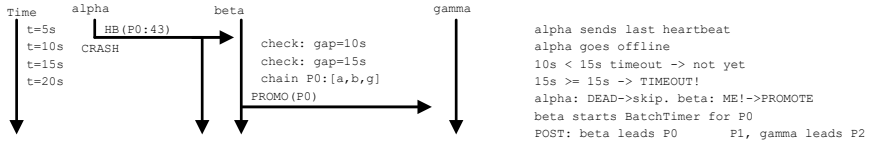
FIG. 12B -- MULTI-LEADER HEARTBEAT FAILOVER: FAILOVER SPLIT-BRAIN CLIENT 1204

SECTION C: FAILOVER DETECTION AND PROMOTION 1204

Timeline: alpha crashes, beta promotes for P0



alpha sends last heartbeat
alpha goes offline
10s < 15s timeout -> not yet
15s >= 15s -> TIMEOUT!
alpha: DEAD->skip. beta: ME!->PROMOTE
beta starts BatchTimer for P0
POST: beta leads P0 P1, gamma leads P2

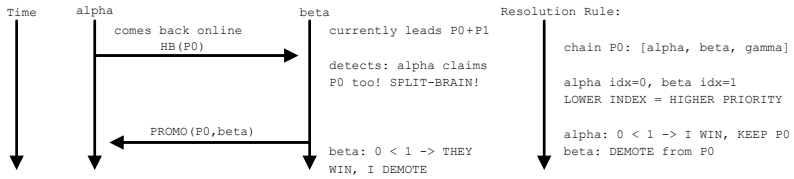
checkHeartbeats Algorithm: promoteToLeader(partIdx, prevLeader, reason):

```
for each (partIdx, entry) in
   leaderMap:
   if entry.isLocalLeader: SKIP
   if (now-lastHB) <= timeout: SKIP
   TIMEOUT -- walk failover chain:
   for candidateId in chain[partIdx]:
      if candidate==deadLeader: skip
      if candidate==me: PROMOTE SELF
      if candidateIsAlive(id): break
         (higher-priority should handle)
      else: continue (also dead)
Each node decides INDEPENDENTLY.
No election messages. Only heartbeat
timestamps       static failover chain.
```

```
1. leaderMap[part].currentLeader=me
    .isLocalLeader=true
2. Start batch timer for partition
3. Emit 'node:started' event
    {action:'leader_promotion'}
4. Broadcast LeaderPromotionPayload
    via 'leader:promotion' gossip
```

SECTION D: SPLIT-BRAIN RESOLUTION 1206

Scenario: alpha recovers while beta already leads P0



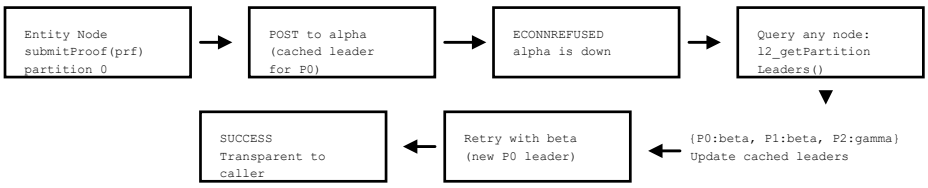
RESOLVED: alpha leads P0 (restored), beta leads P1 (demoted from P0), gamma leads P2

handlePromotion(payload): demoteFromLeader(partIdx):

```
if I ALSO lead this partition:
   chain = failoverChain[partIdx]
   myPri = chain.indexOf(me)
   theirPri = chain.indexOf(them)
   if theirPri < myPri: DEMOTE self
   else: IGNORE (my HB wins)
else: Accept, update leaderMap
```

```
1. Stop batch timer
2. leaderMap[part].isLocalLeader=false
3. Emit 'node:started' event
    {action:'leader_demotion'}
```

SECTION E: CLIENT-SIDE TRANSPARENT FAILOVER 1208



Retry policy: Connection errors only (ECONNREFUSED/ECONNRESET/timeout). Max 1 retry.
DO NOT retry application errors (invalid proof, duplicate receipt, rate limit) -> propagate.